RAG Assignment Report: Baseline Retrieval vs. Cross-Encoder Re-Ranking

Name: Madha Manish
Roll No: 2025202004
GitHub: https://github.com/ManishMjason/RAG-baseline-vs-reranking

Experiment context

I used the UNDP *General Terms and Conditions for Contracts (Goods and/or Services)* as the source document. It is a suitable retrieval test because its rules are spread across 36 numbered articles and include exceptions, qualifications, deadlines, and similar terminology in different clauses.

The document was cleaned and split into 55 clause-aware chunks. The chunks ranged from 45 to 419 tokens, with a mean of 252.6 tokens. The experiment used the following models:

Component	Model or method
Dense Bi-Encoder	`BAAI/bge-small-en-v1.5`
Vector index	FAISS `IndexFlatIP` over normalized 384-dimensional embeddings
Cross-Encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`
Answer generator	`Qwen/Qwen2.5-1.5B-Instruct`

Both pipelines used the same document, chunks, initial Top-10 candidates, answer model, and prompt. The only retrieval difference was whether the initial Top-10 was passed directly to Top-3 selection or re-ranked first.

1. Architecture comparison

A Bi-Encoder processes the query and each document chunk independently. The document embeddings can be computed once and stored in the vector index. At query time, only the query needs to be encoded, after which similarity is calculated between the query vector and every stored chunk vector. In this notebook, the vectors were normalized, so inner product search was equivalent to cosine similarity. This design is fast and reusable, but a single fixed-size vector can lose fine-grained relationships involving negation, conditions, or exceptions.

A Cross-Encoder does not create independent query and chunk vectors. It receives a query and one candidate chunk together and applies self-attention across tokens from both texts. It can therefore model exact word-to-word relationships and subtle constraints more accurately. The trade-off is that it must perform a separate model inference for every query-chunk pair. Its scores also cannot be precomputed for future queries.

The two-stage design combines these strengths. The Bi-Encoder quickly produces ten candidates, and the Cross-Encoder performs the more expensive comparison only on those ten candidates.

2. Qualitative and rank-shift analysis

I evaluated 22 pre-labelled questions. The correct answer-bearing chunk was present in the baseline Top-10 for all 22 questions. Baseline retrieval placed it in the Top-3 for 18 questions, while re-ranking placed it in the Top-3 for 20 questions. Three baseline failures were repaired, although one separate query regressed from the Top-3 after re-ranking.

Failure case	Relevant chunk	Rank shift
Delivery vs. acceptance	`article_11_part_01`	$5 \rightarrow 2$
Subcontractor responsibility	`article_10_part_01`	$4 \rightarrow 2$

Failure case	Relevant chunk	Rank shift
Minimum warranty period	`article_11_part_04`	$4 \rightarrow 3$

The first two queries were used for the controlled Answer 1 versus Answer 2 comparison.

Failure query 1: delivery is not acceptance

The relevant clause states that delivery of goods does not itself constitute acceptance. The baseline placed `article_11_part_01` at rank 5, outside the three chunks sent to the generator. After re-ranking, it moved to rank 2.

Answer 1 still gave a plausible answer: it said UNDP must have a reasonable opportunity to inspect the goods and that payment alone does not constitute acceptance. This happened because another Article 11 chunk in the baseline Top-3 contained related acceptance rules. Answer 2 answered the physical-handover question more directly and its context included the exact delivery clause.

This is therefore better described as degraded evidence selection rather than a complete hallucination. Re-ranking improved direct clause coverage and traceability, even though the baseline generator recovered a reasonable answer from a related passage.

Failure query 2: responsibility for subcontracted work

The direct subcontracting clause, `article_10_part_01`, began at baseline rank 4 and moved to rank 2 after re-ranking. The baseline Top-3 contained only chunks from Article 8 concerning responsibility for personnel. Those chunks had strong semantic overlap with the words *responsibility*, *firm*, and *work*, which acted as distractors.

Both Answer 1 and Answer 2 concluded that responsibility remains with the main contractor. However, only Answer 2 received the direct clause stating that the contractor is solely responsible for services and obligations performed by its subcontractors. The baseline result was semantically plausible, but the advanced result was better grounded in the controlling clause.

Overall retrieval effect

Metric	Baseline	After re-ranking
Top-3 hits	18/22	20/22
Top-3 hit rate	81.8%	90.9%
MRR@10	0.791	0.805
Recall@10 of the initial candidate set	100%	Same candidate set

The improvement was not universal. For the advance-payment security question, the answer-bearing chunk moved from rank 1 to rank 8. This regression is important because it shows that a Cross-Encoder is not guaranteed to improve every query. Across the full test bank, the three repaired cases produced a net gain of two Top-3 hits.

For the two selected answer comparisons, the labelled answer-bearing chunk was absent from both baseline Top-3 contexts and present in both re-ranked Top-3 contexts. Based on those labels, context evidence coverage increased from 0% to 100%. The answer texts themselves changed less dramatically because the generator could infer plausible answers from related baseline chunks.

3. Latency profiling

The following values are means over the 22-query stress-test bank on a Kaggle T4 GPU. Model loading and the one-time indexing operation were excluded from per-query retrieval latency. Creating the 55-vector index took 949.21 ms once.

Pipeline stage	Mean latency per query
Baseline dense retrieval	10.94 ms
Cross-Encoder scoring of the same Top-10	48.87 ms
Advanced retrieval total	59.81 ms

The advanced retrieval stage was about 5.47 times the baseline retrieval time and added 48.87 ms per query. This is a large relative increase at the retrieval stage.

Answer generation was much slower than either retrieval method. Mean generation time was 1543.24 ms for Answer 1 and 1551.43 ms for Answer 2. Across the two selected cases, mean end-to-end time increased from approximately 1553.83 ms to 1608.54 ms, an increase of 54.71 ms or about 3.5%. The retrieval overhead is therefore noticeable in isolation but relatively small compared with local LLM generation.

4. Scaling question

Running a Cross-Encoder over the complete collection from the start would require a full transformer inference for every query-chunk pair. If a collection contained one million chunks, one user query would require approximately one million joint forward passes before an answer could be produced. The computation would grow linearly with the number of chunks, and query-dependent scores could not be precomputed.

The Bi-Encoder avoids this problem because document vectors are calculated once and reused. A query requires one query embedding followed by efficient vector search. At larger scale, an approximate nearest-neighbour FAISS index could reduce search time further, with a possible reduction in candidate recall.

The Cross-Encoder is practical after candidate generation because it scores only a small fixed set, ten pairs in this experiment. Its cost then depends mainly on the number and length of candidates instead of the size of the full document collection. Increasing the candidate count may improve recall, but it also increases re-ranking latency and GPU memory use.

This design has an important limit: the Cross-Encoder can only reorder candidates supplied by the Bi-Encoder. It cannot recover a relevant chunk that is missing from the initial Top-10. Candidate recall therefore places an upper bound on the advanced pipeline.

Conclusion

Cross-Encoder re-ranking improved Top-3 retrieval from 81.8% to 90.9% and promoted three answer-bearing chunks that initially sat at ranks 4-5. For the two analysed queries, the re-ranked context contained the direct controlling clause, while the baseline relied on related passages. The generated baseline answers happened to remain plausible, so the clearest observed benefit was stronger grounding and traceability rather than correction of an obviously wrong answer.

The improvement cost approximately 48.87 ms per query at the retrieval stage and caused one observed regression. I would use this two-stage pipeline when direct evidence quality matters more than minimum retrieval latency. For a latency-sensitive system, I would consider a smaller candidate pool or apply re-ranking only to ambiguous queries.